Procedimientos Almacenados con COMMIT, ROLLBACK

Ejercicio 1: Cree un procedimiento almacenado llamado RegistrarPedido que registre un nuevo pedido en la tabla Orders, recibiendo como parámetros el CustomerID, la OrderDate y el Freight. Antes de insertar el pedido, el procedimiento debe verificar si el cliente existe en la tabla Customers. Si el cliente no existe, se debe lanzar una excepción con un mensaje de error. Si el cliente existe, se debe realizar la inserción dentro de una transacción. En caso de cualquier error durante la operación, se debe hacer un ROLLBACK y mostrar el mensaje de error; en caso contrario, se debe confirmar la transacción con COMMIT.

```
CREATE PROCEDURE RegistrarPedido
  @CustomerID NCHAR(5),
  @OrderDate DATE,
  @Freight MONEY
AS
BEGIN
  BEGIN TRY
    BEGIN TRANSACTION;
      IF NOT EXISTS (SELECT 1 FROM Customers WHERE CustomerID = @CustomerID)
      THROW 50001, 'Cliente no existe.', 1;
      INSERT INTO Orders (CustomerID, OrderDate, Freight)
      VALUES (@CustomerID, @OrderDate, @Freight);
    COMMIT;
  END TRY
  BEGIN CATCH
    ROLLBACK;
    PRINT ERROR_MESSAGE();
  END CATCH
END;
```

Ejercicio 2: Diseñe un procedimiento almacenado denominado TransferirStock que permita transferir una cantidad específica de unidades de inventario desde un producto origen hacia un producto destino, ambos identificados por sus respectivos ProductID. El procedimiento debe verificar que el producto origen tenga suficiente stock disponible antes de realizar la transferencia. Si la validación falla, se debe lanzar una excepción indicando que no hay stock suficiente. Toda la operación debe realizarse dentro de una transacción para garantizar la consistencia de los datos. En caso de éxito, se realiza un COMMIT; si ocurre un error, se ejecuta un ROLLBACK y se muestra el mensaje de error.

```
CREATE PROCEDURE TransferirStock
  @OrigenID INT,
  @DestinoID INT,
  @Cantidad INT
AS
BEGIN
  BEGIN TRY
    BEGIN TRANSACTION;
      DECLARE @StockOrigen INT;
      SELECT @StockOrigen = UnitsInStock FROM Products WHERE ProductID =
@OrigenID;
      IF @StockOrigen < @Cantidad
      THROW 50002, 'Stock insuficiente.', 1;
      UPDATE Products
      SET UnitsInStock = UnitsInStock - @Cantidad
      WHERE ProductID = @OrigenID;
      UPDATE Products
      SET UnitsInStock = UnitsInStock + @Cantidad
      WHERE ProductID = @DestinoID;
    COMMIT;
  END TRY
  BEGIN CATCH
    ROLLBACK;
    PRINT ERROR_MESSAGE();
  END CATCH
END;
```

Ejercicio 3: Implemente un procedimiento almacenado llamado ActualizarPrecioRango que actualice el precio de un producto identificado por su ProductID. El procedimiento debe validar que el nuevo precio (@NuevoPrecio) se encuentre dentro del rango permitido de 10 a 100 unidades monetarias. Si el valor está fuera de este rango, debe lanzar una excepción con un mensaje de error. La operación de actualización debe realizarse dentro de una transacción para asegurar la integridad de los datos, aplicando un COMMIT si todo es correcto o un ROLLBACK en caso de error, mostrando el mensaje correspondiente.

```
CREATE PROCEDURE ActualizarPrecioRango
  @ProductID INT,
  @NuevoPrecio MONEY
AS
BEGIN
  BEGIN TRY
    BEGIN TRANSACTION;

    IF @NuevoPrecio < 10 OR @NuevoPrecio > 100
      THROW 50003, 'Precio fuera de rango.', 1;

    UPDATE Products
    SET UnitPrice = @NuevoPrecio
    WHERE ProductID = @ProductID;

    COMMIT;
  END TRY
  BEGIN CATCH
    ROLLBACK;
    PRINT ERROR_MESSAGE();
  END CATCH
END;
```

Ejercicio 4: Desarrolle un procedimiento almacenado llamado EliminarClienteSeguro que elimine de manera controlada un cliente de la tabla Customers, recibiendo como parámetro el CustomerID. Antes de realizar la eliminación, el procedimiento debe verificar si el cliente tiene pedidos registrados en la tabla Orders. Si existen pedidos asociados, se debe lanzar una excepción que impida la eliminación, mostrando un mensaje de advertencia. Toda la operación debe ejecutarse dentro de una transacción que asegure la integridad de los datos, utilizando COMMIT si se elimina correctamente o ROLLBACK en caso de error.

```
CREATE PROCEDURE EliminarClienteSeguro
  @CustomerID NCHAR(5)
AS
BEGIN
  BEGIN TRY
    BEGIN TRANSACTION;

    IF EXISTS (SELECT 1 FROM Orders WHERE CustomerID = @CustomerID)
      THROW 50004, 'Cliente con pedidos no puede eliminarse.', 1;

    DELETE FROM Customers WHERE CustomerID = @CustomerID;

    COMMIT;
  END TRY
  BEGIN CATCH
    ROLLBACK;
    PRINT ERROR_MESSAGE();
  END CATCH
END;
```

Ejercicio 5: Cree un procedimiento almacenado denominado AsignarEmpleado que permita asignar un empleado a un pedido específico, utilizando los parámetros @OrderID y @EmployeeID. Antes de realizar la asignación, el procedimiento debe validar si el pedido ya tiene un empleado asignado. Si ya existe una asignación previa, se debe lanzar una excepción con un mensaje informativo. La operación debe ejecutarse dentro de una transacción que garantice la consistencia de los datos, aplicando COMMIT si se realiza la actualización correctamente o ROLLBACK en caso de errores

```
CREATE PROCEDURE AsignarEmpleado
  @OrderID INT,
  @EmployeeID INT
AS
BEGIN
  BEGIN TRY
    BEGIN TRANSACTION;

    DECLARE @Asignado INT;
    SELECT @Asignado = EmployeeID FROM Orders WHERE OrderID = @OrderID;

    IF @Asignado IS NOT NULL
      THROW 50005, 'Pedido ya asignado.', 1;

    UPDATE Orders
    SET EmployeeID = @EmployeeID
    WHERE OrderID = @OrderID;

    COMMIT;
  END TRY
  BEGIN CATCH
    ROLLBACK;
    PRINT ERROR_MESSAGE();
  END CATCH
END;
```

Ejercicio 6: Implemente un procedimiento almacenado denominado RegistrarOrdenYDetalle que registre simultáneamente una nueva orden en la tabla Orders y su correspondiente detalle en la tabla [Order Details]. El procedimiento recibe como parámetros el CustomerID, ProductID, la Cantidad y el Precio. Primero, inserta la orden con la fecha actual y obtiene el OrderID generado automáticamente. Luego, utiliza este identificador para insertar el detalle del pedido. Toda la operación debe realizarse dentro de una transacción, aplicando COMMIT si se completa con éxito, o ROLLBACK en caso de errores, utilizando TRY...CATCH para el manejo adecuado de excepciones.

```
CREATE PROCEDURE RegistrarOrdenYDetalle
  @CustomerID NCHAR(5),
  @ProductID INT,
  @Cantidad INT,
  @Precio MONEY
AS
BEGIN
  BEGIN TRY
    BEGIN TRANSACTION;

    DECLARE @OrderID INT;

    INSERT INTO Orders (CustomerID, OrderDate)
    VALUES (@CustomerID, GETDATE());

    SET @OrderID = SCOPE_IDENTITY();

    INSERT INTO [Order Details] (OrderID, ProductID, Quantity, UnitPrice)
    VALUES (@OrderID, @ProductID, @Cantidad, @Precio);

    COMMIT;
  END TRY
  BEGIN CATCH
    ROLLBACK;
    PRINT ERROR_MESSAGE();
  END CATCH
END;
```

Ejercicio 7: Cree un procedimiento almacenado llamado AumentarPrecioCategoria que incremente en un 10% el precio (UnitPrice) de todos los productos pertenecientes a una categoría específica, identificada por el parámetro @CategoryID. Antes de aplicar la actualización, el procedimiento debe verificar que existan productos asociados a dicha categoría. Si no existen productos, se debe lanzar una excepción con un mensaje adecuado. La operación debe ejecutarse dentro de una transacción para asegurar la integridad de los datos, utilizando COMMIT en caso de éxito o ROLLBACK si ocurre un error, gestionado mediante un bloque TRY...CATCH.

```
CREATE PROCEDURE AumentarPrecioCategoria
  @CategoryID INT
AS
BEGIN
  BEGIN TRY
    BEGIN TRANSACTION;

    IF NOT EXISTS (SELECT 1 FROM Products WHERE CategoryID = @CategoryID)
      THROW 50006, 'Categoría sin productos.', 1;

    UPDATE Products
    SET UnitPrice = UnitPrice * 1.10
    WHERE CategoryID = @CategoryID;

    COMMIT;
  END TRY
  BEGIN CATCH
    ROLLBACK;
    PRINT ERROR_MESSAGE();
  END CATCH
END;
```

Ejercicio 8: Implemente un procedimiento almacenado llamado RevertirDescuentos que revise la tabla [Order Details] y restablezca a cero todos los valores de descuento (Discount) que superen el 50%. Antes de aplicar los cambios, el procedimiento debe verificar si existen registros que cumplan dicha condición. Si los hay, realiza la actualización correspondiente. Toda la operación se ejecuta dentro de una transacción para garantizar la consistencia de los datos, utilizando COMMIT al finalizar correctamente o ROLLBACK en caso de errores, los cuales se manejan mediante un bloque TRY...CATCH.

```
CREATE PROCEDURE RevertirDescuentos
AS
BEGIN
  BEGIN TRY
    BEGIN TRANSACTION;

    IF EXISTS (SELECT 1 FROM [Order Details] WHERE Discount > 0.50)
    BEGIN
      UPDATE [Order Details]
      SET Discount = 0
      WHERE Discount > 0.50;
    END

    COMMIT;
  END TRY
  BEGIN CATCH
    ROLLBACK;
    PRINT ERROR_MESSAGE();
  END CATCH
END;
```

Ejercicio 9: Desarrolle un procedimiento almacenado denominado RegistrarProveedor que permita registrar un nuevo proveedor en la tabla Suppliers, recibiendo como parámetros el nombre de la compañía (@CompanyName) y el nombre del contacto (@ContactName). Antes de realizar la inserción, el procedimiento debe validar si ya existe un proveedor con el mismo nombre de compañía. Si existe, se debe lanzar una excepción con un mensaje indicando que el proveedor ya está registrado. La operación debe realizarse dentro de una transacción para asegurar la integridad de los datos, utilizando COMMIT si la inserción es exitosa o ROLLBACK en caso de error, manejando las excepciones mediante un bloque TRY...CATCH.

```
CREATE PROCEDURE RegistrarProveedor
  @CompanyName NVARCHAR(40),
  @ContactName NVARCHAR(30)
AS
BEGIN
  BEGIN TRY
    BEGIN TRANSACTION;

    IF EXISTS (SELECT 1 FROM Suppliers WHERE CompanyName = @CompanyName)
      THROW 50007, 'Proveedor ya existe.', 1;

    INSERT INTO Suppliers (CompanyName, ContactName)
    VALUES (@CompanyName, @ContactName);

    COMMIT;
  END TRY
  BEGIN CATCH
    ROLLBACK;
    PRINT ERROR_MESSAGE();
  END CATCH
END;
```

Ejercicio 10: Implemente un procedimiento almacenado llamado ActualizarDireccion que actualice la dirección (Address) de un cliente específico en la tabla Customers, utilizando el identificador @CustomerID y la nueva dirección proporcionada en el parámetro @NuevaDireccion. Antes de realizar la actualización, el procedimiento debe validar que la dirección no sea nula ni una cadena vacía o compuesta solo por espacios. Si la validación falla, debe lanzarse una excepción con un mensaje de error. Toda la operación debe realizarse dentro de una transacción, usando COMMIT si es exitosa o ROLLBACK en caso de errores, gestionados con un bloque TRY...CATCH.

```
CREATE PROCEDURE ActualizarDireccion
  @CustomerID NCHAR(5),
  @NuevaDireccion NVARCHAR(60)
AS
BEGIN
  BEGIN TRY
    BEGIN TRANSACTION;

    IF @NuevaDireccion IS NULL OR LTRIM(RTRIM(@NuevaDireccion)) = ''
      THROW 50008, 'Dirección inválida.', 1;

    UPDATE Customers
    SET Address = @NuevaDireccion
    WHERE CustomerID = @CustomerID;

    COMMIT;
  END TRY
  BEGIN CATCH
    ROLLBACK;
    PRINT ERROR_MESSAGE();
  END CATCH
END;
```

Ejercicio 11: Cree un procedimiento almacenado llamado CancelarPedido que elimine de forma segura un pedido de la base de datos, identificado por su OrderID. Antes de proceder con la eliminación, el procedimiento debe verificar si el pedido ya ha sido despachado (es decir, si tiene una ShippedDate registrada). Si el pedido ya fue enviado, se debe lanzar una excepción para impedir su cancelación. En caso contrario, se eliminan primero los detalles del pedido de la tabla [Order Details] y luego el pedido de la tabla Orders. Toda la operación debe ejecutarse dentro de una transacción, utilizando COMMIT si se completa con éxito o ROLLBACK en caso de error, controlado mediante TRY...CATCH.

```
CREATE PROCEDURE CancelarPedido
  @OrderID INT
AS
BEGIN
  BEGIN TRY
    BEGIN TRANSACTION;

    IF EXISTS (SELECT 1 FROM Orders WHERE OrderID = @OrderID AND ShippedDate IS NOT NULL)
      THROW 50009, 'Pedido ya despachado.', 1;

    DELETE FROM [Order Details] WHERE OrderID = @OrderID;
    DELETE FROM Orders WHERE OrderID = @OrderID;

    COMMIT;
  END TRY
  BEGIN CATCH
    ROLLBACK;
    PRINT ERROR_MESSAGE();
  END CATCH
END;
```

Ejercicio 12: Desarrolle un procedimiento almacenado llamado ClasificarPedidosFlete que clasifique los pedidos registrados en la tabla Orders según el valor del flete (Freight) en tres categorías: Alto, Medio o Bajo. Antes de realizar la clasificación, el procedimiento debe verificar si existen pedidos con valores nulos en el campo Freight. En caso de encontrar al menos uno, debe lanzar una excepción informando sobre la inconsistencia. Si no hay valores nulos, se ejecuta una consulta que devuelve el OrderID, el Freight y su respectiva categoría. Toda la operación debe estar contenida en una

transacción, con COMMIT si la ejecución es exitosa o ROLLBACK en caso de error, controlado mediante TRY...CATCH.

```
CREATE PROCEDURE ClasificarPedidosFlete
AS
BEGIN
  BEGIN TRY
    BEGIN TRANSACTION;

    IF EXISTS (SELECT 1 FROM Orders WHERE Freight IS NULL)
      THROW 50010, 'Pedidos con flete nulo.', 1;

    SELECT OrderID, Freight,
        CASE
          WHEN Freight >= 100 THEN 'Alto'
          WHEN Freight >= 50 THEN 'Medio'
          ELSE 'Bajo'
        END AS Categoria
    FROM Orders;

    COMMIT;
  END TRY
  BEGIN CATCH
    ROLLBACK;
    PRINT ERROR_MESSAGE();
  END CATCH
END;
```